

將 Python 2 App Engine Cloud NDB 和 Cloud Tasks 應用程式遷移至 Python 3 和 Cloud Datastore (模組 9)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-9-py3dstasks?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將瞭解如何將 Python 2 App Engine Cloud NDB 和 Cloud Tasks (v1) 應用程式遷移至 Python 3、Cloud Datastore 和 Cloud Tasks (v2)

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定](#)
5. [5. 更新應用程式檔案](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室的目的是將第 8 課的範例應用程式移植到 Python 3，並將 Datastore (Datastore 模式下的 Cloud Firestore) 存取權從 [Cloud NDB](#) 切換至原生 [Cloud Datastore](#) 用戶端程式庫，以及升級至最新版本 [Cloud Tasks](#) 用戶端程式庫。

我們在第 7 模組中新增了 Task Queue 的使用方式，以處理 **push** 工作，然後在第 8 模組中將該使用方式遷移至 Cloud Tasks。在第 9 個單元中，我們將繼續介紹 Python 3 和 Cloud Datastore。使用工作佇列處理**提取**工作的開發人員，將遷移至 Cloud Pub/Sub，並應改為參閱第 18 至 19 節。

未使用工作佇列？

如果您的應用程式未使用 App Engine 工作佇列服務，請略過第 7 到 9 節 (推送工作) 和第 18 到 19 節 (提取工作)，或將這些程式碼研究室當做練習，熟悉工作佇列遷移作業。

在接下來的研究室中

- 將第 8 堂課的範例應用程式移植到 Python 3
- 將 Datastore 存取權從 Cloud NDB 切換至 Cloud Datastore 用戶端程式庫
- 升級至最新版 Cloud Tasks 用戶端程式庫

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 8 單元 App Engine 應用程式：完成[第 8 單元程式碼研究室](#) (建議)，或從存放區複製[第 8 單元應用程式](#)

問卷調查

2. 背景

[第 7 個單元](#)說明如何在 Python 2 Flask App Engine 應用程式中使用 App Engine 工作佇列發送工作。在[單元 8](#)中，您會將該應用程式從工作佇列遷移至 Cloud Tasks。在[第 9 堂課](#)中，您將繼續這趟旅程，將應用程式移植到 Python 3，並將 Datastore 存取作業從使用 [Cloud NDB](#) 切換為原生 [Cloud Datastore](#) 用戶端程式庫。

由於 Cloud NDB 適用於 Python 2 和 3，因此對於將應用程式從 Python 2 移植到 3 的 App Engine 使用者來說，這就足夠了。將用戶端程式庫遷移至 Cloud Datastore 是**完全選用**的程序，只有在**一個情況下**才需要考慮：您有非 App Engine 應用程式 (和/或 Python 3 App Engine 應用程式) 已使用 Cloud Datastore 用戶端程式庫，且想將程式碼集整合為只用一個用戶端程式庫存取 Datastore。Cloud NDB 專為 Python 2 App Engine 開發人員而建，是 Python 3 的遷移工具，因此如果您還沒有使用 Cloud Datastore 用戶端程式庫的程式碼，就不需要考慮這項遷移作業。

最後，Cloud Tasks 用戶端程式庫的開發作業只會在 Python 3 中繼續進行，因此我們將從其中一個最終 Python 2 版本「遷移」至其 Python 3 同類版本。幸好，Python 2 沒有重大變更，因此您不需要採取任何行動。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

本節將說明如何：

1. 設定 Cloud 專案
2. 取得基準範例應用程式
3. (重新) 部署及驗證基準應用程式

這些步驟可確保您從可運作的程式碼開始，並準備好遷移至雲端服務。

1. 設定專案

如果您已完成[第 8 堂課的程式碼研究室](#)，請重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案具備可用的帳單帳戶和已啟用的 App Engine 應用程式。找出專案 ID，因為在本程式碼研究室中，每當遇到 `PROJECT_ID` 變數時，您都需要使用該 ID。

2. 取得基準範例應用程式

其中一項必要條件是可正常運作的第 8 模組 App Engine 應用程式：完成第 8 模組程式碼研究室 (建議)，或從存放區複製第 8 模組應用程式。無論您使用自己的程式碼還是我們的程式碼，我們都會從第 8 模組的程式碼開始 (「START」)。本程式碼研究室會逐步說明如何遷移，最後的程式碼會與第 9 模組存放區資料夾 (「FINISH」) 中的程式碼類似。

- 開始：[第 8 個模組的存放區](#)
- 完成：[模組 9 存放區](#)
- [整個存放區](#) (複製或下載 ZIP 檔)

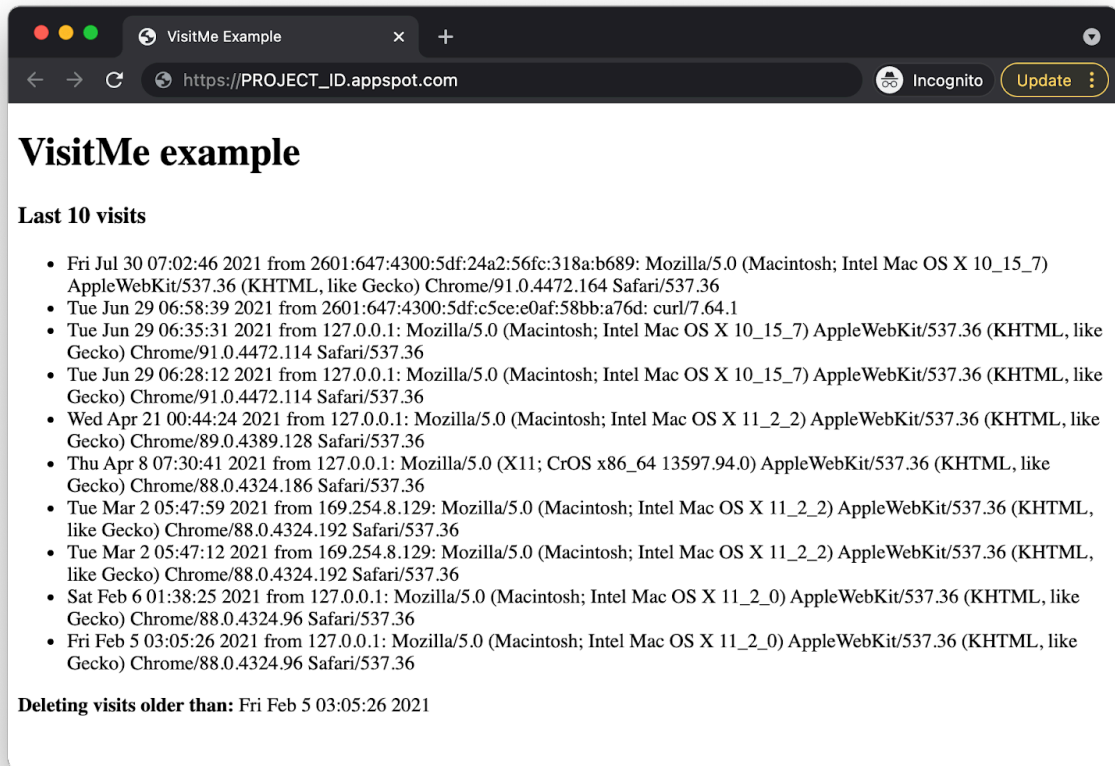
無論您使用哪個第 7 模組應用程式，資料夾都應如下所示，可能也會有 `lib` 資料夾：

```
$ ls
README.md          appengine_config.py  requirements.txt
app.yaml            main.py              templates
```

3. (重新) 部署及驗證基準應用程式

請按照下列步驟部署第 8 課的應用程式：

1. 刪除 `lib` 資料夾 (如有)，然後執行 `pip install -t lib -r requirements.txt` 重新填入 `lib`。如果開發電腦同時安裝 Python 2 和 3，可能需要改用 `pip2`。
2. 請確認您已[安裝及初始化](#) `gcloud` 指令列工具，並查看[其用法](#)。
3. (選用) 使用 `gcloud config set project PROJECT_ID` 設定 Cloud 專案，這樣您就不必在發出的每個 `gcloud` 指令中輸入 `PROJECT_ID`。
4. 使用 `gcloud app deploy` 部署範例應用程式
5. 確認應用程式是否正常運作。如果您已完成第 8 堂程式碼研究室，應用程式會顯示造訪次數最多的訪客，以及最近的造訪記錄 (如下圖所示)。底部會顯示即將刪除的舊工作。



步驟 4 · 約 2 分鐘

4. 更新設定

requirements.txt

新的 `requirements.txt` 與第 8 堂課的幾乎相同，只有一個重大變更：將 `google-cloud-ndb` 換成 `google-cloud-datastore`。進行這項變更後，您的 `requirements.txt` 檔案應如下所示：

```
flask
google-cloud-datastore
google-cloud-tasks
```

這個 `requirements.txt` 檔案沒有任何版本號碼，表示系統會選取最新版本。如果發生任何不相容問題，標準做法是使用版本號碼，為應用程式鎖定可正常運作的版本。

app.yaml

第二代 App Engine 執行階段不支援內建第三方程式庫 (如 2.x)，也不支援複製非內建程式庫。第三方套件的唯一條件是必須列在 `requirements.txt` 中。因此可以刪除 `app.yaml` 的整個 `libraries` 區段。

另一項更新是，Python 3 執行階段需要使用自行進行路由的網頁架構。因此，所有指令碼處理常式都必須變更為 `auto`。不過，由於所有路徑都必須變更為 `auto`，且這個範例應用程式不會提供任何靜態檔案，因此擁有任何處理常式都無關緊要，請一併移除整個 `handlers` 區段。

您只需要在 `app.yaml` 中將執行階段設為支援的 Python 3 版本，例如 3.10。進行這項變更，讓新的縮寫 `app.yaml` 只有這一行：

```
runtime: python310
```

刪除 appengine_config.py 和 lib

新一代 App Engine 執行階段會大幅調整第三方套件的使用方式：

- 內建程式庫是由 Google 審查並在 App Engine 伺服器上提供，可能是因為內含開發人員不得部署至雲端的 C/C++ 程式碼。這類程式庫已不再適用於第 2 代執行階段。
- 在第 2 代執行階段中，不再需要複製非內建程式庫 (有時稱為「供應商」或「自行套裝組合」)。而應列在 `requirements.txt` 中，建構系統會在部署時自動為您安裝。

由於第三方套件管理系統有所變更，因此不需要 `appengine_config.py` 檔案和 `lib` 資料夾，請將其刪除。在第 2 代執行階段中，App Engine 會自動安裝 `requirements.txt` 中列出的第三方套件。摘要：

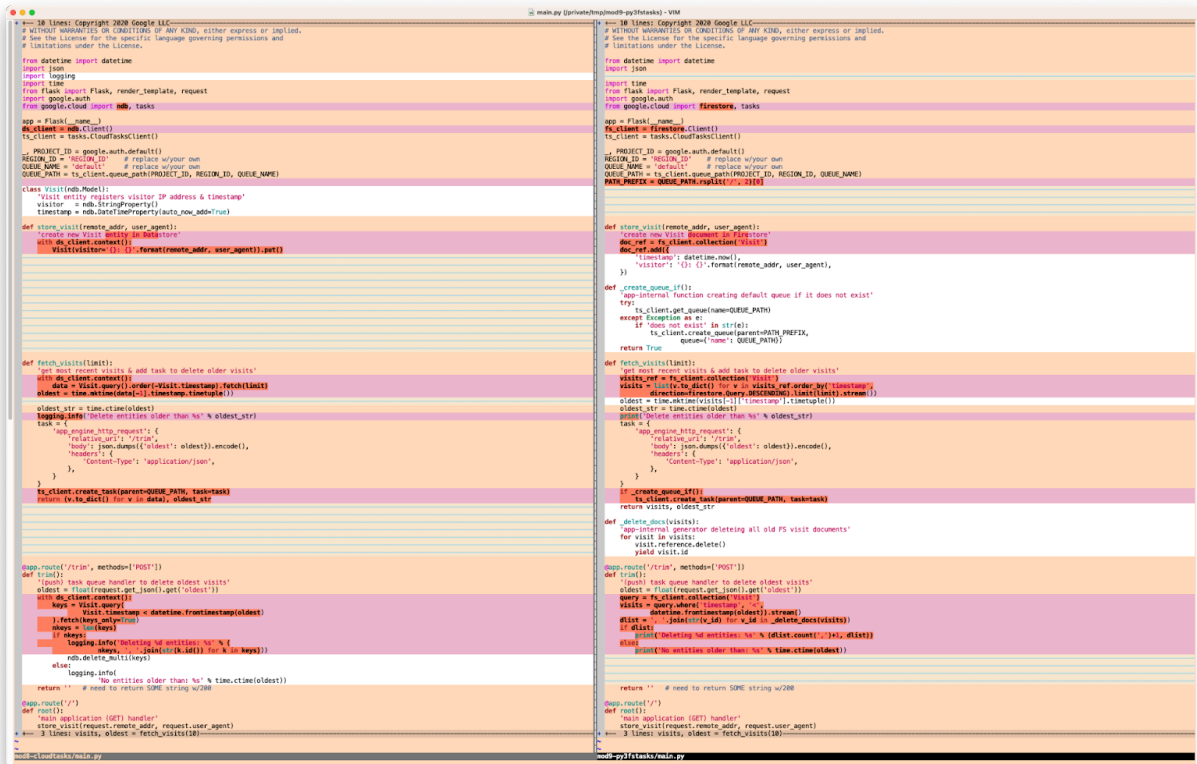
- 不得自行組合或複製第三方程式庫，請在 `requirements.txt` 中列出這些程式庫
- 沒有 `pip install` 進入 `lib` 資料夾，也就是沒有 `lib` 資料夾週期
- `app.yaml` 中沒有內建第三方程式庫的清單 (因此沒有 `libraries` 區段)，請在 `requirements.txt` 中列出
- 應用程式不需要參照任何第三方程式庫，因此不需要 `appengine_config.py` 檔案

開發人員只需在 `requirements.txt` 中列出所有需要的第三方程式庫。

步驟 5 · 約 23 分鐘

5. 更新應用程式檔案

只有一個應用程式檔案 `main.py`，因此本節中的所有變更只會影響該檔案。下圖是「差異」說明，顯示將現有程式碼重構至新應用程式時，需要進行的整體變更。讀者不必逐行閱讀程式碼，因為這張圖的目的只是以圖像方式概略說明重構作業的需求 (但如果需要，可以開啟新分頁或下載圖片並放大)。



更新匯入作業和初始化

單元 8 的 `main.py` 匯入部分使用 Cloud NDB 和 Cloud Tasks，應如下所示：

BEFORE:

```
from datetime import datetime
import json
import logging
import time
from flask import Flask, render_template, request
import google.auth
from google.cloud import ndb, tasks

app = Flask(__name__)
ds_client = ndb.Client()
ts_client = tasks.CloudTasksClient()
```

在 Python 3 等第二代執行階段中，記錄功能經過簡化和強化：

- 如要享有完整的記錄體驗，請使用 [Cloud Logging](#)
- 如要簡單記錄，只要透過 `print()` 將訊息傳送至 `stdout` (或 `stderr`) 即可

- 不需要使用 Python `logging` 模組 (因此請移除)

因此，請刪除 `logging` 的匯入內容，並將 `google.cloud.ndb` 換成 `google.cloud.datastore`。同樣地，請將 `ds_client` 變更為指向 Datastore 用戶端，而非 NDB 用戶端。完成這些變更後，新應用程式頂端會如下所示：

修改後：

```
from datetime import datetime
import json
import time
from flask import Flask, render_template, request
import google.auth
from google.cloud import datastore, tasks

app = Flask(__name__)
ds_client = datastore.Client()
ts_client = tasks.CloudTasksClient()
```

遷移至 Cloud Datastore

現在，請將 NDB 用戶端程式庫用法替換為 Datastore。App Engine NDB 和 Cloud NDB 都需要資料模型 (類別)，這個應用程式的資料模型是 `Visit`。`store_visit()` 函式在所有其他遷移模組中的運作方式都相同：建立新的 `visit` 記錄，儲存訪客用戶端的 IP 位址和使用者代理程式 (瀏覽器類型)，藉此註冊造訪。

BEFORE:

```
class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor = ndb.StringProperty()
    timestamp = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{ }: {}'.format(remote_addr, user_agent)).put()
```

不過，Cloud Datastore **不會**使用資料模型類別，因此請刪除該類別。此外，Cloud Datastore 不會在建立記錄時自動建立時間戳記，因此您必須手動建立，這項作業是透過 `datetime.now()` 呼叫完成。

如果沒有資料類別，修改後的 `store_visit()` 應如下所示：

修改後：

```
def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    entity = datastore.Entity(key=ds_client.key('Visit'))
    entity.update({
        'timestamp': datetime.now(),
        'visitor': '{ }: {}'.format(remote_addr, user_agent),
    })
    ds_client.put(entity)
```

主要功能是 `fetch_visits()`。這項作業不僅會針對最新的 `visit` 執行原始查詢，還會擷取上次顯示的 `visit` 時間戳記，並建立呼叫 `/trim` (因此為 `trim()`) 的推送工作，大量刪除舊的 `visit`。以下是使用 Cloud NDB 的範例：

BEFORE:

```
def fetch_visits(limit):
    'get most recent visits & add task to delete older visits'
    with ds_client.context():
        data = Visit.query().order(-Visit.timestamp).fetch(limit)
        oldest = time.mktime(data[-1].timestamp.timetuple())
        oldest_str = time.ctime(oldest)
        logging.info('Delete entities older than %s' % oldest_str)
        task = {
            'app_engine_http_request': {
                'relative_uri': '/trim',
                'body': json.dumps({'oldest': oldest}).encode(),
                'headers': {
                    'Content-Type': 'application/json',
                },
            }
        }
        ts_client.create_task(parent=QUEUE_PATH, task=task)
    return (v.to_dict() for v in data), oldest_str
```

主要異動：

1. 將 Cloud NDB 查詢換成 Cloud Datastore 對等項目，查詢樣式略有不同。
2. Datastore 不像 Cloud NDB，不需要使用內容管理工具，也不會要求您擷取資料 (使用 `to_dict()`)。
3. 以 `print()` 取代記錄呼叫

這些變更完成後，`fetch_visits()` 看起來會像這樣：

修改後：

```
def fetch_visits(limit):
    'get most recent visits & add task to delete older visits'
    query = ds_client.query(kind='Visit')
    query.order = ['-timestamp']
    visits = list(query.fetch(limit=limit))
    oldest = time.mktime(visits[-1]['timestamp'].timetuple())
    oldest_str = time.ctime(oldest)
    print('Delete entities older than %s' % oldest_str)
    task = {
        'app_engine_http_request': {
            'relative_uri': '/trim',
            'body': json.dumps({'oldest': oldest}).encode(),
            'headers': {
                'Content-Type': 'application/json',
            },
        }
    }
    ts_client.create_task(parent=QUEUE_PATH, task=task)
    return visits, oldest_str
```

通常這樣就足夠了。但很遺憾，這項功能有一個重大問題。

(可能) 建立新的 (發送) 佇列

在單元 7 中，我們將 App Engine `taskqueue` 的使用方式新增至現有的單元 1 應用程式。將推送工作做為舊版 App Engine 功能的一項主要優點，就是系統會自動建立「預設」佇列。在第 8 模組中，該應用程式遷移至 Cloud Tasks 時，預設佇列已存在，因此我們當時仍不需要擔心。這項做法在單元 9 中有所改變。

請務必注意，新的 App Engine 應用程式不再使用 App Engine 服務，因此您無法再假設 App Engine 會在其他產品 (Cloud Tasks) 中自動建立工作佇列。如上所述，在 `fetch_visits()` 中建立工作 (適用於不存在的佇列) 會失敗。您需要新函式來檢查「default」佇列是否存在，如果不存在，則建立一個。

將此函式命名為 `_create_queue_if()`，並新增至 `fetch_visits()` 正上方，因為這是呼叫函式的位置。要新增的函式主體：

```
def _create_queue_if():
    'app-internal function creating default queue if it does not exist'
    try:
        ts_client.get_queue(name=QUEUE_PATH)
    except Exception as e:
        if 'does not exist' in str(e):
            ts_client.create_queue(parent=PATH_PREFIX,
                                   queue={'name': QUEUE_PATH})
    return True
```

Cloud Tasks `create_queue()` 函式需要佇列的完整路徑名稱，但不包括佇列名稱。為求簡單，請建立另一個變數 `PATH_PREFIX`，代表 `QUEUE_PATH` 減去佇列名稱 (`QUEUE_PATH.rsplit('/', 2)[0]`)。在頂端附近新增其定義，讓包含所有常數指派項目的程式碼區塊如下所示：

```
_, PROJECT_ID = google.auth.default()
REGION_ID = 'REGION_ID'      # replace w/your own
QUEUE_NAME = 'default'       # replace w/your own
QUEUE_PATH = ts_client.queue_path(PROJECT_ID, REGION_ID, QUEUE_NAME)
PATH_PREFIX = QUEUE_PATH.rsplit('/', 2)[0]
```

現在請修改 `fetch_visits()` 中的最後一行，使用 `_create_queue_if()`，視需要先建立佇列，然後再建立工作：

```
if _create_queue_if():
    ts_client.create_task(parent=QUEUE_PATH, task=task)
return visits, oldest_str
```

現在 `_create_queue_if()` 和 `fetch_visits()` 應如下所示：

```
def _create_queue_if():
    'app-internal function creating default queue if it does not exist'
    try:
        ts_client.get_queue(name=QUEUE_PATH)
    except Exception as e:
        if 'does not exist' in str(e):
            ts_client.create_queue(parent=PATH_PREFIX,
                                   queue={'name': QUEUE_PATH})
    return True

def fetch_visits(limit):
    'get most recent visits & add task to delete older visits'
    query = ds_client.query(kind='Visit')
    query.order = ['-timestamp']
    visits = list(query.fetch(limit=limit))
    oldest = time.mktime(visits[-1]['timestamp'].timetuple())
    oldest_str = time.ctime(oldest)
    print('Delete entities older than %s' % oldest_str)
    task = {
        'app_engine_http_request': {
            'relative_uri': '/trim',
            'body': json.dumps({'oldest': oldest}).encode(),
            'headers': {
                'Content-Type': 'application/json',
            },
        }
    }
    if _create_queue_if():
        ts_client.create_task(parent=QUEUE_PATH, task=task)
    return visits, oldest_str
```

除了必須加入這段額外程式碼外，其餘 Cloud Tasks 程式碼大多與第 8 模組相同。最後要查看的程式碼是工作處理常式。

更新 (推送) 工作處理常式

在工作處理常式 `trim()` 中，Cloud NDB 程式碼會查詢比顯示的舊記錄更早的瀏覽記錄。這項功能會使用僅限鍵的查詢來加快速度，因為如果只需要造訪 ID，就不必擷取所有資料。取得所有造訪 ID 後，請使用 Cloud NDB 的 `delete_multi()` 函式，以批次方式刪除所有 ID。

BEFORE:

```
@app.route('/trim', methods=['POST'])
def trim():
    '(push) task queue handler to delete oldest visits'
    oldest = float(request.get_json().get('oldest'))
    with ds_client.context():
        keys = Visit.query(
            Visit.timestamp < datetime.fromtimestamp(oldest)
        ).fetch(keys_only=True)
        nkeys = len(keys)
        if nkeys:
            logging.info('Deleting %d entities: %s' % (
                nkeys, ', '.join(str(k.id()) for k in keys))
            )
            ndb.delete_multi(keys)
        else:
            logging.info(
                'No entities older than: %s' % time.ctime(oldest))
    return '' # need to return SOME string w/200
```

與 `fetch_visits()` 類似，大部分的變更都涉及將 Cloud NDB 程式碼換成 Cloud Datastore、調整查詢樣式、移除其內容管理工具的使用，以及將記錄呼叫變更為 `print()`。

修改後：

```
@app.route('/trim', methods=['POST'])
def trim():
    '(push) task queue handler to delete oldest visits'
    oldest = float(request.get_json().get('oldest'))
    query = ds_client.query(kind='Visit')
    query.add_filter('timestamp', '<', datetime.fromtimestamp(oldest))
    query.keys_only()
    keys = list(visit.key for visit in query.fetch())
    nkeys = len(keys)
    if nkeys:
        print('Deleting %d entities: %s' % (
            nkeys, ', '.join(str(k.id) for k in keys))
        )
        ds_client.delete_multi(keys)
    else:
        print('No entities older than: %s' % time.ctime(oldest))
    return '' # need to return SOME string w/200
```

主要應用程式處理常式 `root()` 沒有任何變更。

移植到 Python 3

這個範例應用程式的設計可同時在 Python 2 和 3 上執行。本教學課程相關章節已說明 Python 3 的專屬變更。不需要額外步驟或相容性程式庫。

Cloud Tasks 更新

支援 Python 2 的 Cloud Tasks 用戶端程式庫最終版本為 1.5.0。撰寫本文時，Python 3 的最新版用戶端程式庫與該版本完全相容，因此不需要進一步更新。

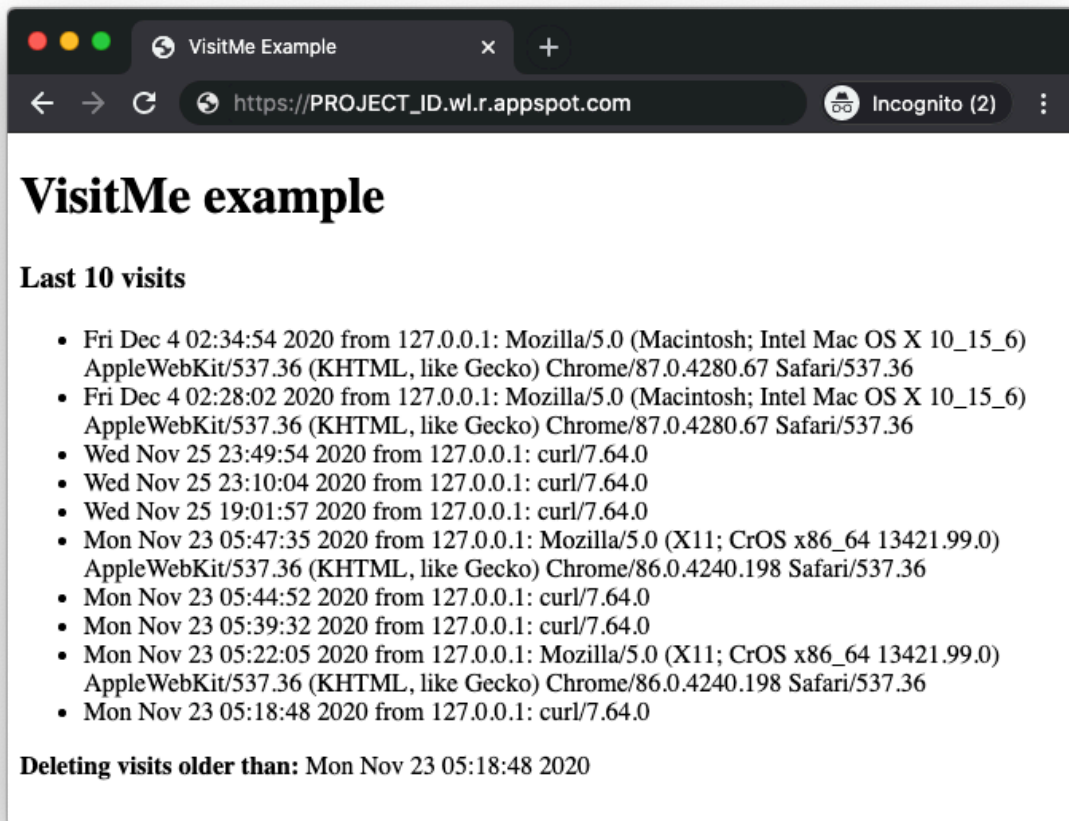
更新 HTML 範本

HTML 範本檔案 `templates/index.html` 也不需要任何變更，因此這會完成所有必要變更，以取得第 9 堂課的應用程式。

6. 摘要/清除

部署及驗證應用程式

完成程式碼更新 (主要是移植到 Python 3) 後，請使用 `gcloud app deploy` 部署應用程式。輸出內容應與第 7 和 8 堂課的應用程式相同，但您已將資料庫存取權移至 Cloud Datastore 用戶端程式庫，並升級至 Python 3：



這個步驟是程式碼研究室的最後一個步驟。歡迎比較您的程式碼與 [Module 9 資料夾](#) 中的程式碼。恭喜！

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`

- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `*LOCATION`，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- Cloud Tasks 提供免費方案，詳情請參閱[定價頁面](#)。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

至此，我們已完成從 App Engine 工作佇列發送工作遷移至 Cloud Tasks 的作業。[第 3 堂課](#)也會介紹如何從 Cloud NDB 遷移至 Cloud Datastore (不使用 Task Queue 或 Cloud Tasks)。除了第 3 節之外，還有其他遷移模組著重於從 App Engine 舊版服務套裝組合遷移，包括：

- [單元 2](#)：從 App Engine NDB 遷移至 Cloud NDB
- [第 3 堂課](#)：從 Cloud NDB 遷移至 Cloud Datastore
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage
- [第 18-19 堂課](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

7. 其他資源

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 8 堂課 (START) 和第 9 堂課 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
範本 8	code	(不適用)
單元 9	(不適用)	code

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- App Engine [定價](#)和[配額](#)資訊

Cloud NDB

- [Google Cloud NDB 說明文件](#)
- [Google Cloud NDB 存放區](#)
- [Cloud Datastore 定價資訊](#)

Cloud Datastore

- [Google Cloud Datastore 說明文件](#)
- [Google Cloud Datastore 存放區](#)
- [Cloud Datastore 定價資訊](#)

Cloud Tasks

- [Google Cloud Tasks 說明文件](#)

- [Google Cloud Tasks 存放區](#)
- [Cloud Tasks 定價資訊](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK](#) (`gcloud` 指令列工具)
- [所有 Google Cloud 說明文件](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
